Differenza e implementazione di No Throw, Basic e Strong Guarantee.

Nel contesto delle eccezioni in C++, le tre garanzie principali sono "No-throw guarantee", "Basic guarantee" e "Strong guarantee", che definiscono il livello di garanzia fornito dalla funzione in caso di errori.

La "No-throw guarantee" garantisce che una funzione non lancerà alcuna eccezione. Questa garanzia si applica quando si lavora con oggetti immutabili e si evitano operazioni rischiose come l'allocazione di memoria dinamica o la chiamata di funzioni che potrebbero generare un'eccezione. Ad esempio, la funzione std::sqrt() garantisce la "No-throw guarantee" perché non lancia mai un'eccezione.

```
void print(const std::string& s) noexcept {
   std::cout << s << std::endl;
}
```

La "Basic guarantee" garantisce che, se un'eccezione viene lanciata, lo stato degli oggetti coinvolti sarà consistente, ma non necessariamente lo stato originale. Ad esempio, se una funzione aggiunge un elemento a una lista, la "Basic guarantee" garantisce che la lista non sarà danneggiata, ma potrebbe non contenere l'elemento aggiunto. Può prevedere l'uso di try…catch per fare in modo che la funzione abbia successo o lanci un'eccezione senza aver avuto memory leak.

```
#include <vector>
#include <stdexcept>

// Esempio di funzione con Basic guarantee
void push_back(std::vector<int>& v, int value) {
  try {
    v.push_back(value);
  }
  catch (const std::exception& e) {
    // Se viene lanciata un'eccezione, lo stato del vettore sarà consistente
    // ma non conterrà il valore inserito
    std::cerr << "push_back failed: " << e.what() << '\n';
  }
}
```

La "Strong guarantee" garantisce che se un'eccezione viene lanciata, lo stato degli oggetti coinvolti rimarrà invariato. Dunque, oltre alla basic guarantee, la funzione fa in modo che tutti i valori non locali alla funzione siano gli stessi dopo l'eccezione di quello che erano alla chiamata della funzione. Ad esempio, se una funzione modifica il contenuto di una lista, la "Strong guarantee" garantisce che la lista rimarrà invariata in caso di errore.

```
// Esempio di funzione con Strong guarantee
void remove(std::vector<int>& v, int value) {
  auto it = std::find(v.begin(), v.end(), value);
  if (it == v.end()) {
    throw std::out_of_range("value not found");
  }

  try {
    v.erase(it);
  }
  catch (const std::exception& e) {
    // Se viene lanciata un'eccezione, lo stato del vettore sarà lo stesso di prima
    // perché la funzione ha effettuato un rollback delle modifiche
```

```
    std::cerr << "remove failed: " << e.what() << '\n';
  }
}
```

Eccezioni

std::exception • La standard library fornisce una varietà di eccezioni • Tutte le eccezioni nella standard library derivano da std::exception • Si possono dividere in due famiglie: • std::logic_error • Riporta un errore che consegue una logica difettata come la violazione di pre-condizioni o gli invarianti della classe • std::runtime_error • Riporta errori dovuti a eventi esterni al programma stesso che sono difficilmente prevedibili

Design della classe std::exception

```
class exception {
 public: exception();
         exception(const exception&);
         exception& operator=(const exception&);
         virtual ~exception();
         virtual const char* what() const;
 };
```

Derivare dalle standard exceptions • Piuttosto che creare una propria classe per l'eccezioni, è dunque meglio: • Utilizzare la libreria standard (std::runtime_error, std::out_of_range…) • Creare una classe derivata dalla libreria standard class My_error : runtime_error { public: My_error(int x) : error_value{x} {} int error_value; const char* what() const override { return "My_error"; } };

Definizione ed esempio del problema dello slicing e possibili strategie di risoluzione.

Lo slicing è un comportamento insolito che si verifica a seguito dell'assegnamento o copia di un oggetto di una classe derivata ad una variabile di classe base non virtuale. Durante l'assegnamento, infatti, avviene la copia dei soli membri della classe base, con la conseguente perdita di informazioni legate ai membri appartenenti alla sola classe derivata. Ciò si verifica perché l'operazione di copia o assegnamento invocano il costruttore di copia o l'operatore di assegnamento della classe base, che non sono in grado di copiare le informazioni specifiche dell'oggetto derivato. Per risolvere tale problema è necessario disabilitare il costruttore di copia e l'operatore di assegnamento di copia della classe base utilizzando la parola chiave "=delete" dopo la dichiarazione, oppure dichiarandoli come private. In alternativa si può evitare la copia dell'oggetto derivato, andando ad assegnare un riferimento all'oggetto derivato ad un puntatore alla classe base.

Tuttavia, è necessario che la classe derivata fornisca una propria implementazione del costruttore di copia e dell'operatore di assegnazione di copia.

```
#include <iostream>

class Base {
public:
   virtual void print() const {
      std::cout << "I am a Base object" << std::endl;
   }
};
```

```
class Derived : public Base {
public:
   virtual void print() const {
      std::cout << "I am a Derived object" << std::endl;
   }
};

int main() {
   Derived d;
   Base b = d;
   b.print(); // output: I am a Base object, invece di I am a derived object
   return 0;
}
```

Definizione ed esempio di dangling pointer e possibili strategie di risoluzione del problema.

Il dangling pointer è un comportamento in C++ che si verifica quando l'oggetto puntato da un puntatore è stato distrutto ma il puntatore non è uscito dallo scope in cui è definito ed è pertanto accessibile Di conseguenza, l'accesso a una posizione di memoria deallocata può comportare comportamenti indefiniti del programma, difficili da individuare e debuggare. Per risolvere tale problema, è necessario assegnare nullptr al puntatore dopo la deallocazione della memoria oppure l'utilizzo di reference al posto di puntatori quando possibile, oppure l'assegnazione di un nuovo indirizzo di memoria al puntatore prima di utilizzarlo.

```
#include <iostream>

int* createArray(int size) {
  int* arr = new int[size];
  return arr;
}

int main() {
  int* arr = createArray(10);
  delete[] arr;

  std::cout << arr[0] << std::endl;

  return 0;
}
```

Differenza tra static_cast, reinterpret_cast e dynamic_cast con esempi

Reinterpret_cast() è un operatore di conversione in grado di reinterpretare un pattern di bit passati come parametro, che considera quindi i bit di un valore come rappresentazione di un altro tipo. Si tratta di un'operazione potenzialmente pericolosa, perché non garantisce la corretta interpretazione dei dati.

```
int x = 42;
char* p = reinterpret_cast<char*>(&x);
std::cout << static_cast<void*>(p) << '\n';
```

static_cast è un operatore di cast in C++ che consente di convertire un'espressione di un tipo in un altro tipo. Viene utilizzato per convertire tra tipi correlati, come conversioni numeriche tra tipi di dati, conversioni di puntatore tra tipi di classe o conversioni da puntatori a void a puntatori ad altri tipi. È un cast verificato a tempo di compilazione, il che significa che la compilazione si interrompe se la conversione non è possibile.

```
double d = 3.14159;
int i = static_cast<int>(d);
std::cout << "d: " << d << std::endl; // output: d: 3.14159
std::cout << "i: " << i << std::endl; // output: i: 3
```

Dymanic_cast è un operatore di conversione di tipo che permette di convertire in modo sicuro e a tempo di esecuzione un puntatore o riferimento ad un oggetto di classe base in un puntatore o riferimento ad un oggetto di classe derivata. C++ mette a disposione due operatori per effettuare queste ultime conversioni:
Dynamic_cast<D*>(p), dato come parametro un riferimento ad un oggetto di classe base, restituisce un puntatore ad oggetto di classe derivata D. Se la conversione non va a buon fine, la funzione restituisce un puntatore impostato a 0.
Dynamic_cast<D&>(*p), dato come parametro un puntatore ad un oggetto di classe base, restituisce un riferimento ad oggetto di classe derivata D. Se la conversione non va a buon fine, la funzione lancia un'eccezione di tipo std::bad_cast.

```
class Base {};
class Derived : public Base {};

int main() {
  Base* p = new Derived;
  Derived* dp = dynamic_cast<Derived*>(p);
  if (dp) {
    std::cout << "Conversion to Derived* successful" << std::endl;
  } else {
    std::cout << "Conversion to Derived* failed" << std::endl;
  }

  try {
    Derived& dr = dynamic_cast<Derived&>(*p);
    std::cout << "Conversion to Derived& successful" << std::endl;
  } catch (const std::bad_cast& e) {
    std::cout << "Conversion to Derived& failed" << std::endl;
  }

  return 0;
}
```

Scrittura di una classe con costruttore di copia movimento e assegnazione di copia e movimento, costruttore e distruttore

```
vector::vector(const vector& arg)
    : sz{arg.sz}, elem{new double[arg.sz]}
{
    std::copy(arg.elem, arg.elem+sz, elem);
}
```

NEW DOUBLE ESSERE USATO NON CONOSCO LA DIM. DELL'ARRAY

```
vector& vector::operator=(const vector& a)
{
    double* p = new double[a.sz];    // 1. alloca nuovo spazio
    copy(a.elem, a.elem+a.sz, p);    // 2. copia gli elementi
    delete[] elem;                   // 3. de-alloco vecchio spazio
    elem = p;                        // 4. resetto puntatore
    sz = a.sz;                       // 5. resetto dimensione
    return *this;                    // 6. ritorno self-reference
}
```

```
vector::vector(vector&& a) : sz{a.sz}, elem{a.elem} // ruba il puntatore ad a
{
    a.sz = 0;                    // annulla a
    a.elem = nullptr;            // invalida il puntatore di a
}

vector& vector::operator=(vector&& a)
{
    delete[] elem;               // dealloca lo spazio
    elem = a.elem;               // riassegna il puntatore
    sz = a.sz;                   // riassegno il size
    a.elem = nullptr;            // invalida il vecchio puntatore di a
    a.sz = 0;                    // azzera il size di a
    return *this;                // ritorna una self-reference
}
```

Esempio e definizione di friend function e friend class

In C++, una friend function o friend class è una funzione o una classe esterna ad una classe che ha il permesso di accedere ai membri privati e protetti di quella classe.

```
#include <iostream>

class MyClass {
private:
  int private_member = 10;
  friend void print_private_member(MyClass obj); // friend function
  friend class FriendClass; // friend class

public:
  int public_member = 20;
};

class FriendClass {
public:
  void print_private_member(MyClass obj) {
    std::cout << "MyClass::private_member: " << obj.private_member << std::endl;
  }
};
```

```
void print_private_member(MyClass obj) {
  std::cout << "MyClass::private_member: " << obj.private_member << std::endl;
};


int main() {
  MyClass obj;
  FriendClass friend_obj;
  friend_obj.print_private_member(obj);
  print_private_member(obj);
  return 0;
}
```

È potenzialmente pericoloso perché consente a funzioni e classi esterne alla classe di modificare i membri di quest'ultima, violando l'incapsulamento e rendendo più difficile il debug in programmi complessi.

Overloading di operatori con esempi

L'overloading di operatori permette di modificare o definire l'effetto degli operatori sugli oggetti. Per esempio, se è necessario effettuare la somma di un data member di un oggetto "Esempio" con lo stesso dato presente in un altro oggetto di tipo "Esempio" è necessario effettuare l'overloading di +, in quanto non è definito di default l'effetto che l'operatore ha sull'oggetto. L'overloading avviene in questo modo:

```
class Esempio{

public:

Esempio operator+( const Esempio& e1, const Esempio& e2) {

   Esempio result;

   result->data = e1->data += e2->data;

}

}
```

La funzione membro così definita deve ovviamente ritornare un oggetto di tipo Esempio, in quanto ci si aspetta che l'operazione "Esempio e = e1 + e2" restituisca un oggetto di tale tipo. Gli argomenti sono di tipo Esempio anch'essi e viene usata la parola chiave const in quanto non devono essere modificati, inoltre sono passati per riferimento in quanto è inutile che vengano passati per copia gli interi oggetti. Viene poi creato l'oggetto che verrà restituito e il suo valore di data sarà equivalente alla somma dei data member "data" degli oggetti e1 ed e2.

Definizione ed utilizzo di namespace

Un namespace è un raggruppamento di dichiarazioni di classi, variabili, funzioni. Permette di organizzare il codice in modo gerarchico ed evitare i clash tra scope. Viene definito con la notazione namespace name{}. Per utlizzare in un programma un membro appartenente ad un namespace occorre utlizzare l'operatore :: e scrivere un fully qualified name (es. MyNameSpace::var) oppure è possibile importare un namespace in

```
#include <iostream>
#include <string>
using namespace std;
namespace A
{
    int var = 10;
}
namespace B
{
    int cout = 5;
}
int main()
{
    using namespace B;
    cout << A::var;
    return 0;
}
```

una classe o funzione. Se vengono però importati due namespace con membri che hanno lo stesso nome, il compilare segnala un errore, definendo l'ambiguità nell'uso di un membro.

Differenza tra vari scope ed implementazione di ciascuno scope a partire da una variabile data

Lo scope di una variabile è una regione di testo del codice in cui tale variabile è accessibile e visibile. In C++ sono presenti i seguenti scope: local scope, che rappresenta l'area di testo all'interno del corpo di una funzione e la lista degli argomenti di quest'ultima; global scope, che rappresenta l'area di testo fuori da qualsiasi altro scope e rende una variaible accessibile in qualsiasi punto del programma; statement scope, ossia l'area di testo all'interno di uno statement come if, for, while, switch; namespace scope, l'area di testo all'interno di un namespace; class scope, ossia l'area di testo all'interno di una classe.

```
#include <iostream>

int global_var = 10;

namespace MyNamespace {
  int global_var = 20;

  class MyClass {
  public:
    int member_var = 30;

    void print(int local_var) {
      int global_var = 40;
      std::cout << "local_var: " << local_var << std::endl; //50
      std::cout << "MyClass::member_var: " << member_var << std::endl; //30
      std::cout << "MyNamespace::global_var: " << MyNamespace::global_var << std::endl; //20
      std::cout << "global_var: " << ::global_var << std::endl; //30
            std::cout << global_var << std::endl; //40
    }
  };
}

int main() {
  int local_var = 50;
  MyNamespace::MyClass obj;
  obj.print(local_var);
  return 0;
}
```

Descrizione degli effetti di vari modificatori di accesso

In C++, i modificatori di accesso vengono utilizzati per specificare l'accessibilità dei membri delle classi da parte di altri componenti del programma. Ci sono tre modificatori di accesso disponibili:

public: i membri dichiarati come pubblici sono accessibili ovunque da qualsiasi parte del programma. Questo significa che qualsiasi funzione può accedere ai membri pubblici della classe.

private: i membri dichiarati come privati sono accessibili solo all'interno della classe stessa. Ciò significa che le funzioni esterne alla classe non possono accedere ai membri privati della classe.

protected: i membri dichiarati come protetti sono accessibili solo all'interno della classe stessa e dalle sue classi derivate. Ciò significa che le funzioni esterne alla classe e non derivate non possono accedere ai membri protetti della classe.

L'uso dei modificatori di accesso serve a garantire l'incapsulamento, ovvero la separazione tra l'implementazione interna della classe e il suo utilizzo da parte del resto del programma. In questo modo, si può garantire la corretta funzionalità della classe e proteggere gli utenti della classe da modifiche accidentali ai membri interni.

Differenza tra Array style c e standard Vector con esempi

In C++, gli array stile C e i vettori (vector) sono due strutture dati utilizzate per contenere un insieme di elementi dello stesso tipo.

Gli array stile C sono strutture dati statiche di lunghezza fissa. Per accedere ad un elemento specifico dell'array, si utilizza un indice intero che indica la posizione dell'elemento nell'array. Gli array stile C sono dichiarati scrivendo “int[] a {0,1,2};”. Poiché gli array stile C hanno lunghezza fissa, non è possibile aggiungere o rimuovere elementi dall'array. Inoltre, se si cerca di accedere ad un elemento fuori dal limite dell'array, si verifica un comportamento indefinito, che potrebbe causare errori di runtime.

I vettori (vector) sono strutture dati dinamiche che possono contenere un numero variabile di elementi. I vettori sono implementati come template della libreria standard di C++ e possono essere dichiarati come segue: “std::vector<int> myVector;” I vettori sono simili agli array stile C, ma hanno alcune differenze importanti: i vettori possono crescere e diminuire di dimensione durante l'esecuzione del programma, aggiungendo o rimuovendo elementi in coda o in posizioni specifiche dell'array. Per accedere agli elementi del vettore, si utilizza la sintassi [] come per gli array stile C, ma i vettori includono un controllo di indice a tempo di esecuzione per evitare gli errori di accesso fuori dal limite dell'array.

I vettori includono una vasta gamma di metodi della libreria standard per manipolare gli elementi all'interno del vettore, come ad esempio push_back, pop_back, insert, erase, resize, clear, swap, e molti altri. In generale, i vettori sono una scelta più flessibile e sicura rispetto agli array stile C. Tuttavia, se la dimensione dell'array è fissa e nota a priori, gli array stile C possono essere più efficienti in termini di spazio e di accesso ai dati.

Spazio occupato da variabili locali e globali in memoria. Differenza tra text segment, data segment e uninitialized data segment

Text Segment, Initialized Data Segment e Uninitialized Data Segment sono tre segmenti di memoria distinti che vengono utilizzati per memorizzare dati in un programma C++ durante l'esecuzione.

Text Segment è una porzione di memoria che contiene il codice sorgente compilato del programma. Il codice include le istruzioni, le funzioni, le procedure e i sottoprogrammi del programma.

Initialized Data Segment è una porzione di memoria che contiene i dati inizializzati del programma, come le variabili globali e le variabili statiche che sono state assegnate un valore durante la compilazione.

Uninitialized Data Segment: Il Uninitialized Data Segment è una porzione di memoria che contiene le variabili globali e le variabili statiche che non sono state inizializzate durante la compilazione. Questi dati verranno inizializzati con valori predefiniti dal sistema operativo durante l'esecuzione del programma.

Le variabili locali, se non definite statiche, sono invece inserite all'interno di record di attivazione della funzione che le contiene all'interno dello stack. Lo stack rappresenta una porzione di memoria destinata alla memorizzazione temporana dei dat che, oltre alle variabili locali, contiene argomenti di funzioni, e puntatori ai dati. La creazione di un record di attivazione avviene ogni qualvolta la funzione viene invocata durante l'esecuzione del programma. La distruzione delle variabili locali definite nella funzione avviene al termine dell'esecuzione del blocco di codice corrispondente. È importante notare che le variabili locali non hanno bisogno di essere esplicitamente allocate o deallocate dall'utente.

Descrivi le tre fasi del processo di compilazione C++.

La prima fase è quella effettuata dal preprocessore che si occupa di elaborare le direttive del preprocessore, come #include e #define, sostituendo queste ultime con il loro contenuto eliminando i commenti. Infine, Invia al compilatore un nuovo codice sorgente da compilare.

Durante la fase di compilazione avviene invece l'analisi sintattica e semantica del codice Se non sono presenti errori di compilazione, il compilatore C++ genera il codice macchina eseguibile corrispondente al codice sorgente andando a creare diversi file oggetto con estensione .o. Nella fase di linking ultima fase del processo di compilazione, il linker unisce tutti i file oggetto generati durante la compilazione e risolve tutti i riferimenti esterni al codice legati a librerie di sistema o librerie di terze parti.

Definire l'aritmetica dei puntatori illustrando: 1) il concetto alla base dell'aritmetica; 2) con quali operatori e operandi funziona; 3) due esempi di funzionamento, ciascuno corredato da codice che deve essere commentato mediante descrizione e/o grafici.

I puntatori in C++ sono un meccanismo che permette di salvare in una variabile un indirizzo di memoria. Questo significa che è possibile conoscere la posizione in memoria dei dati a cui si associa un puntatore, e successivamente accedervi. Con aritmetica dei puntatori si intende la modalità con cui si può accedere ai dati salvati in zone di memoria contigua semplicemente incrementando o decrementando il valore di un puntatore.

I puntatori supportano l'utilizzo di molti operatori: ++, +, += e i corrispondenti operatori con il segno – hanno la funzione di incrementare (o decrementare) la posizione in memoria, l'indirizzo di memoria viene modificato in base alla lunghezza del tipo di dato che viene puntato. Es. se il dato che viene puntato dal puntatore ha lunghezza 20, ed il puntatore viene incrementato di 3, l'effettivo indirizzo viene spostato di 20*3. E' supportato anche l'operatore * che permette di dereferenziare un puntatore, o in parole più semplici accedere al contenuto che è presente in memoria all'indirizzo contenuto. L'operatore & viene utilizzato per ottenere la posizione in memoria di una variabile o oggetto (int* p = &a;), ed infine gli operatori == e != vengono utilizzati per verificare che due puntatori puntino allo stesso indirizzo.

Il codice seguente stampa il contenuto di un array tramite lo scorrimento di un puntatore.

```
int[] a {0,1,2,3,4,5};
```

```
int* p = a; // punta l'inizio dell'array
for(int i = 0; i<a.length();i++){
   std::cout << *p;//ritorna il contenuto puntato da p
   p++;
}
```

Il codice seguente modifica la variabile a tramite l'utilizzo di un puntatore.

```
std::string a ("Esempio");
std::string* p = &a;
*p = "modificata";
std::cout << a;//stamperà "modificata"
```

Definire il concetto di polimorfismo. Descrivere il polimorfismo dinamico e il polimorfismo statico evidenziandone le differenze e gli strumenti implementativi che il C++ offre a supporto. Mostrare un esempio di codice di classe che sfrutti il polimorfismo dinamico e un esempio di classe che sfrutti il polimorfismo statico.

Il polimorfismo è uno dei pilastri della programmazione orientata agli oggetti in C++. È definito come l'abilità di associare comportamenti diversi ad un'unica notazione. In C++ si possono distinguere due tipi di polimorfismo.

Il polimorfismo statico è anche detto a tempo di compilazione ed è fortemente legato al concetto di programmazione generica. È implementato tramite l'uso di template, ossia un meccanismo che favorisce il riutilizzo del codice e ne evita la duplicazione andando a definire funzioni e classi che possono essere usati per diversi tipi di dato, inclusi quelli definiti dall'utente. Per definire una funzione o una classe template è necessario far precedere queste ultime dalla dichiarazione template <typename T>. In questo caso, il compilatore genera la classe con il tipo effettivo che verrà usato al posto del parametro template. Lo svantaggio legato al polimorfismo statico, e più precisamente all'uso dei template riguarda la poca separazione tra l'interno del template (la sua definizione) e l'interfaccia.

Dato che la definizione del template deve essere inclusa nel file header, che contiene anche l'interfaccia della classe o della funzione, ogni volta che viene utilizzata una classe o una funzione basata su un template, l'intero codice del template deve essere ricompilato, anche se solo uno o due parametri del template sono stati modificati.

```
//esempio di classe template
#include <iostream>

template<typename TID, typename TGrade>
class Student {
public:
  Student(TID id, TGrade grade) : id_(id), grade_(grade) {}
```

```
  void print() const {
    std::cout << "ID: " << id_ << ", media voti: " << grade_ << std::endl;
  }
private:
  TID id_;
  TGrade grade_;
};

// esempio di funzione template
template <typename T>
T const& max(T const& a, T const& b) {
    return a < b ? b : a;
}

int main() {
  Student<int, double> s1(1234, 25.5);
  s1.print();

  Student<std::string, int> s2("5678", 30);
  s2.print();
    // utilizzo della funzione template max
    int a = 3, b = 5;
    std::cout << "max: " << max(a, b) << std::endl;

    return 0;
}
```

Il polimorfismo dinamico è chiamato anche polimorfismo a tempo di esecuzione ed è implementato tramite ereditarietà e funzioni virtuali.

Descrivere le caratteristiche fondamentali degli smart pointers. Illustrare le differenze fra unique_ptr<T> e shared_ptr<T>. Descrivere il comportamento delle seguenti funzioni della classe unique_ptr<T>: reset() e release().

Gli smart pointer sono oggetti che gestiscono automaticamente la memoria allocata per gli oggetti, in modo da evitare problemi come la perdita di memoria (memory leak) e la deallocazione di memoria già deallocata (dangling pointers).

I principali tipi di smart pointer in C++ sono unique_ptr<T>, shared_ptr<T> che, per essere utilizzati richiedono l'inclusione della libreria memory.

unique_ptr<T> è un puntatore intelligente che possiede un oggetto di tipo T e garantisce che l'oggetto venga distrutto quando il puntatore esce dallo scope in cui è stato creato. Non è possibile copiare un unique_ptr<T> poiché l'oggetto può avere una sola proprietà, ma è possibile invece effettuare move (spostamento dell'oggetto contenuto nel puntatore). Un'altra restrizione è data dal fatto che due unique_ptr non possono puntare allo stesso oggetto.

shared_ptr<T> è un puntatore intelligente che possiede un oggetto di tipo T, ma consente più copie dello stesso puntatore. A differenza di unique_ptr, quindi permette le copie e la condivisione dell'oggetto puntato. Il conteggio dei riferimenti viene utilizzato per determinare quando l'oggetto deve essere deallocato. Quando il conteggio dei riferimenti raggiunge zero, l'oggetto viene deallocato automaticamente.
Sia unique pointer, sia shared ptr fanno uso dell'operatore di dereferencing per consentire l'accesso all'oggetto puntato, e l'operatore -> per usare i membri del tipo di oggetto contenuto nel puntatore.

Le funzioni della classe unique_ptr<T> sono:

reset(): distrugge l'oggetto corrente puntato dal puntatore, sostituendolo con un nuovo oggetto di tipo T se fornito come parametro.
release(): rilascia il controllo dell'oggetto gestito dal puntatore e restituisce il puntatore a tale oggetto, senza distruggerlo.

```
#include <memory>
#include <iostream>
#include <string>

class MyClass {
public:
  MyClass(const std::string& name) : name_(name) {
    std::cout << "Constructor called for " << name_ << std::endl;
  }
  ~MyClass() {
    std::cout << "Destructor called for " << name_ << std::endl;
  }
  void hello() {
    std::cout << "Hello from " << name_ << std::endl;
  }
private:
  std::string name_;
};

int main() {
  // Unique pointer example
  std::unique_ptr<MyClass> ptr1 = std::make_unique<MyClass>("object1");
  ptr1->hello();
  std::unique_ptr<MyClass> ptr2 = std::move(ptr1);
  ptr2->hello();
  ptr2.reset();

  // Shared pointer example
  std::shared_ptr<MyClass> ptr3 = std::make_shared<MyClass>("object2");
  std::shared_ptr<MyClass> ptr4 = ptr3;
  ptr3->hello();
  ptr4->hello();
  ptr3.reset();

  return 0;
```

}

Definire il concetto di ereditarietà. Descrivere approfonditamente gli strumenti disponibili in C++ per implementare tale concetto. Descrivere le differenze fra le relazioni “is a” e “has a” fra classi e illustrare tali differenze con un breve esempio di codice.

L'ereditarietà è un concetto fondamentale della programmazione orientata agli oggetti che consente di definire gerarchie di classi. In particolare, permette di definire classi derivate a partire da una classe base, in modo tale che la classe derivata erediti tutti i membri della classe base e possa accedere a tutti i membri della classe base denominati come protected o public. Inoltre, si ha che la classe derivata può sovrascrivere le funzioni della classe base e può avere funzioni e membri aggiuntivi.
In C++ l’ereditarietà viene implementata tramite classi derivate, funzioni virtuali e classi astratte, consentendo di fatto l’implementazione del polimorfismo a tempo di esecuzione, ossia l’abilità di associare comportamenti specifici diversi ad un’unica notazione.

Per definire una classe derivata in C++, si utilizza la seguente sintassi:

```
class DerivedClass : accessSpecifier BaseClass {
  // ...
};
```

dove accessSpecifier può essere public, protected o private a seconda della visibilità dei membri ereditati.

Una funzione virtuale è una funzione di classe base che può essere sovrascritta dalle classi che le ereditano. Si contraddistingue per la presenza della parola chiave virtual nella firma della dichiarazione. Se la funzione è virtuale pura, le classi derivate devono assolutamente sovrascrivere tale funzione, pena errore di compilazione. La parola chiave ovveride esplicita l’overriding di una funzione virtuale all’interno di una classe derivata.

In C++, una classe astratta è una classe che non può essere istanziata direttamente e contiene almeno una funzione virtuale pura. L'utilizzo di una classe astratta consente di definire un'interfaccia comune che deve essere implementata dalle sottoclassi, ma senza dover specificare l'implementazione concreta delle funzioni. In questo modo, è possibile scrivere del codice che lavora con oggetti di tipo classe astratta senza sapere esattamente di quale sottoclasse si tratta, ma solo che implementa le funzioni richieste dalla classe astratta.

```
#include <iostream>

class Animal {
public:
   virtual void speak() {
      std::cout << "Animals cannot speak." << std::endl;
   }
};

class Dog : public Animal {
public:
   void speak() override {
```

```
        std::cout << "Woof!" << std::endl;
    }
};

int main() {
    Animal* animal;

    Dog dog;
    animal = &dog;
    animal->speak();

    return 0;
}
```

In questo modo, abbiamo realizzato il polimorfismo grazie alle funzioni virtuali: la funzione speak() viene eseguita sulla classe derivata corretta, a seconda dell'oggetto a cui il puntatore si riferisce.

La relazione “is a” indica il rapporto tra una classe base e una sua derivata. Nell’esempio sopra, si ha che l’oggetto di classe derivata Dog è un oggetto di classe base Animal, ossia possiede tutti i membri della classe base.

La relazione "has a" si riferisce alla composizione tra classi in cui un'istanza di una classe contiene uno o più membri di un'altra classe. Questa relazione definisce l'aggregazione o la composizione tra le classi, in cui un'istanza della classe "contenitore" (quella che "ha" l'oggetto) detiene un'istanza della classe "contenuto".

```
class Computer {
public:
  void turnOn() {
    std::cout << "Computer turned on" << std::endl;
  }
};

class Monitor {
public:
  void turnOn() {
    std::cout << "Monitor turned on" << std::endl;
  }
};

class ComputerMonitor {
private:
  Computer computer;
  Monitor monitor;

public:
  void turnOn() {
    computer.turnOn();
    monitor.turnOn();
```

```
  }
};
```

In questo esempio, abbiamo una classe Computer che rappresenta un computer e una classe Monitor che rappresenta un monitor. La classe ComputerMonitor combina queste due classi utilizzando la composizione, il che significa che un oggetto ComputerMonitor ha un oggetto Computer e un oggetto Monitor.

Approfondimento su Layout di Memoria che consente Ereditarietà

Il layout degli oggetti in C++ rappresenta come le varie parti dell'oggetto sono organizzate in memoria. In particolare, quando si utilizzano classi derivate, ci sono alcune componenti che devono essere gestite in modo specifico.

Una delle componenti importanti è il virtual pointer. Il virtual pointer è un puntatore a una virtual table (vtable), che è una tabella contenente i puntatori alle funzioni virtuali della classe. Questo puntatore viene aggiunto al layout dell'oggetto solo se la classe ha almeno una funzione virtuale. Il puntatore virtuale viene collocato all'inizio dell'oggetto, immediatamente prima dei membri della classe.

La virtual table contiene le funzioni virtuali della classe in ordine di dichiarazione e alcune informazioni di tipo, come il tipo della classe stessa. Ogni classe con funzioni virtuali ha la sua virtual table.

In una classe derivata, il layout degli oggetti include anche un puntatore alla virtual table della classe base. Questo viene aggiunto alla fine dell'oggetto derivato, subito dopo i membri della classe. In questo modo, la classe derivata eredita il puntatore alla virtual table della classe base e può accedere alle funzioni virtuali della classe base attraverso di esso.

L'utilizzo del puntatore alla virtual table garantisce che, durante l'esecuzione del programma, il codice corretto per una funzione virtuale venga chiamato anche in presenza di polimorfismo.

2) Descrivere l'allocazione dinamica della memoria in C++, spiegando che effetti ha sul layout della memoria. Riportare e descrivere gli strumenti di programmazione in C++ utilizzati per l'allocazione dinamica. Riportare un esempio di codice di gestione dinamica della memoria all'interno di una classe.

In C++, la gestione dell'allocazione dinamica della memoria può essere effettuata attraverso l'utilizzo degli operatori "new" e "delete". L'operatore "new" viene utilizzato per allocare dinamicamente la memoria, mentre "delete" viene utilizzato per deallocarla. Ad esempio, per allocare dinamicamente un intero, si può utilizzare il seguente codice "int * p = new int;". In questo caso, l'operatore "new" alloca dinamicamente la memoria necessaria per un intero e restituisce un puntatore a tale area di memoria. Poiché l'allocazione dinamica della memoria è gestita dal sistema operativo, la memoria allocata dinamicamente non si trova nello stack della funzione corrente, ma viene invece allocata in una zona di memoria chiamata heap. Inoltre, poiché la memoria allocata dinamicamente non viene deallocata automaticamente come avviene per le variabili locali dello stack, la zona di memoria heap può diventare più grande man mano che il programma alloca più memoria.

Per deallocare la memoria, si usa l'operatore "delete". È importante notare che l'allocazione dinamica della memoria deve essere effettuata con attenzione, poiché una cattiva gestione della memoria può portare a problemi di sicurezza e a malfunzionamenti del programma. Inoltre, la memoria allocata dinamicamente deve essere sempre deallocata quando non è più necessaria per evitare perdite di memoria (memory leaks).

```
class DynamicIntArray {
private:
  int* array;
   int size;
public:
   DynamicIntArray(int size) {
      array = new int[size];//il puntatore array punta all’inizio della nuova memoria allocata
      this->size = size;
   }
   ~DynamicIntArray() {
      delete[] array;//nel distruttore la zona di memoria viene deallocata
   }
   int& operator[](int index) {
      if (index >= size) {
         // gestione dell'errore: l'indice è fuori dal range dell'array
         throw std::out_of_range("Index out of range");
      }
      return array[index];
   }
};
```

Esempi di memory leak si verificano quando:

Quando si utilizza una variabile puntatore e si dimentica di deallocare la memoria con l'operatore "delete".

Quando si utilizza una variabile puntatore per allocare memoria, ma si sovrascrive il puntatore senza deallocare la memoria precedentemente allocata, causando una perdita di memoria.

Quando si utilizza un ciclo per allocare memoria, ma non si libera la memoria allocata una volta che il ciclo è terminato.

Quando si utilizza una funzione per allocare memoria, ma non si restituisce un puntatore alla memoria allocata o si restituisce un puntatore a una memoria non allocata, causando una perdita di memoria.

3) Illustrare il concetto di predicato e fornire un esempio in C++. Spiegare in che modo un predicato può essere usato in un algoritmo STL, corredando la spiegazione con un esempio di codice.

In C++, un predicato è una funzione o un oggetto funzione che ritorna un valore booleano, utilizzato per valutare una proprietà di un oggetto o di un insieme di oggetti. Per creare un predicato, C++ mette a disposizione del programmatore lambda expressions e function object.

Una function object, d'altra parte, è un oggetto che si comporta come una funzione, ovvero può essere invocato utilizzando l'operatore di chiamata funzione operator(). Un function object può essere definito come una classe con l'operatore operator() definito, oppure può essere creato utilizzando una lambda expression.

Una lambda expression è una tecnica che consente di scrivere funzioni in-line senza nome. Una lambda expression (o lambda function) è un modo per: Definire un function object, Crearne uno immediatamente e usarlo come argomento di una funzione

• Tale function object è anonimo e non utilizzabile altrove
Tali funzioni possono catturare varabili locali con l'operatore[] per utilizzarle al loro interno. L'operatore () di overloading delle lambda expression è const di default, cioè non può modificare variabili catturate a meno che non siano passate per riferimenti. Se posto come mutable, le variabili locali catturate dalla lambda expression possono essere modificate.

```
#include <iostream>
#include <vector>
#include <algorithm>

bool is_odd(int n) {
  return n % 2 != 0;
}

int main() {
  std::vector<int> vec = {2, 4, 7, 8, 10};
  auto it = std::find_if(vec.begin(), vec.end(), is_odd);
  if (it != vec.end()) {
    std::cout << "Il primo numero dispari è: " << *it << std::endl;
  } else {
    std::cout << "Non ci sono numeri dispari nel vettore" << std::endl;
  }
  return 0;
}
```

In questo esempio, la funzione is_odd è il predicato utilizzato dall'algoritmo std::find_if per valutare se un dato numero è dispari. std::find_if scorre il vettore utilizzando il predicato per ogni elemento, e restituisce l'iteratore che punta al primo elemento che soddisfa la condizione del predicato (in questo caso, essere dispari). Se nessun elemento soddisfa la condizione, std::find_if restituisce l'iteratore di fine del vettore.

1) Illustrare il concetto di libreria in C++. Spiegare la differenza tra librerie statiche e librerie dinamiche, evidenziando per ciascuna tipologia quali sono i punti di forza e di debolezza.

Una libreria in C++ è un insieme di funzioni, variabili, e classi precompilati. Il vantaggio derivato dall'uso di librerie è legato al riutilizzo del codice in più progetti e alla semplificazione dello sviluppo software. Le librerie statiche sono file oggetto compilati da cui il linker, durante la compilazione, estrae il codice necessario (funzioni invocate, classi e variabili utilizzate nel programma) e le copia nell'eseguibile finale. Ciò che si ottiene è un file eseguibile che tiene una copia del codice della libreria statica, non può essere spezzato. Le librerie statiche garantiscono dunque buona portabilità, ma aumentano notevolmente le dimensioni dell'eseguibile finale, e non vengono condivise da più programmi. Al contrario, le librerie dinamiche sono file oggetto compilati caricati in memoria durante l'esecuzione del programma. Durante in fase di compilazione, infatti, il linker crea solamente una connessione tra i file oggetto del programma e il codice della libreria. In fase di esecuzione, è il linker dinamico ad occuparsi di fornire al programma puntatori alle funzioni/classi/variabili richieste. Questa pratica consente a più programmi di utilizzare contemporaneamente la stessa libreria dinamica senza la necessità di mantenerne una copia in memoria, garantisce file eseguibili di dimensioni minori (rispetto all'uso di librerie statiche), e possono essere ricompilate senza la necessità di modificare gli eseguibili.

```
ar rvs lib/libmy_func_static.a my_func.o
```

```
g++ -shared -o lib/libmy_func_dynamic.so my_func.o
```

a. Definire i concetti di sequenza, contenitore e iteratore della Standard Template Library (STL), descrivendone le caratteristiche fondamentali. Illustrare i principi con cui tali concetti sono sviluppati, mostrando come si raggiunge l'accesso uniforme ai dati. Spiegare come sono gestiti gli iteratori begin e end. Fornire brevi esempi di codice per illustrare i concetti descritti.

La Standard Template Library (STL) di C++ definisce tre concetti fondamentali: sequenza, contenitore e iteratore.

Una sequenza è una collezione di elementi ordinati, che possono essere acceduti uno alla volta.
Un contenitore è una struttura dati che contiene un insieme di elementi e supporta le operazioni di copia, chiamando il tipo dei propri elementi value_type.
Un iteratore è un oggetto che fornisce un modo per accedere agli elementi di una sequenza o di un contenitore.
Gli elementi di una sequenza o di un contenitore sono memorizzati in una struttura dati interna, che può essere implementata in vari modi, ad esempio con un array dinamico, una lista concatenata, un albero binario, ecc. Tuttavia, l'accesso agli elementi è sempre uniforme, grazie all'uso degli iteratori.

Gli iteratori sono oggetti che consentono di accedere agli elementi di una sequenza o di un contenitore in modo indipendente dalla struttura interna. Ogni sequenza o contenitore definisce due iteratori: uno che punta all'inizio della sequenza o del contenitore (iteratore begin), e uno che punta alla fine (iteratore end). Gli iteratori vengono utilizzati per scorrere la sequenza o il contenitore, accedere agli elementi, inserire o rimuovere elementi, ecc.

```
#include <iostream>
#include <vector>

int main() {
   std::vector<int> v = {1, 2, 3, 4, 5};
   for (auto it = v.begin(); it != v.end(); ++it) {
      std::cout << *it << " ";
   }
   std::cout << std::endl;
   return 0;
}
```

In questo esempio, la classe vector rappresenta il contenitore, mentre gli iteratori begin() e end() vengono utilizzati per scorrere la sequenza. L'operatore * viene utilizzato per accedere al valore dell'elemento puntato dall'iteratore.

Il vantaggio di utilizzare gli iteratori è che il codice diventa indipendente dalla struttura interna del contenitore.

Garanzie della Standard Template Library

Accesso uniforme ai dati • Indipendente da come sono salvati • Indipendente dal tipo usato • Accesso safe • Facilità di visita alle strutture dati • Salvataggio compatto dei dati • Aggiunta, rimozione e ricerca veloci • Disponibilità degli algoritmi più comuni • copia, trova, cerca, ordina, somma

Disaccoppiamento lato algoritmo: non è necessario conoscere i dettagli della struttura dati; è sufficiente usare i relativi iteratori (Es: un algoritmo di ordinamento può operare pur senza conoscere i dettagli della struttura dati)
Disaccoppiamento lato struttura dati: non è necessario esporre tutti i dettagli e gestire tutti i modi d'uso; è sufficiente implementare ed esporre gli iteratori (Es: un vector non ha necessità di conoscere l'algoritmo che opera su di esso per permettergli di funzionare; è sufficiente che esponga gli iteratori)

Iteratori vs. puntatori • Concetto di iteratore legato a quello di puntatore • Infatti un puntatore a un elemento di un array è un iteratore • Ma molti iteratori non sono puntatori • Gli iteratori possono definire ad esempio il range-check • Gli iteratori garantiscono una flessibilità e generalità che i puntatori non hanno

b. Illustrare i contenitori vector e list, spiegando quali sono le differenze.

In C++, vector e list sono entrambi contenitori sequenziali di elementi, ma sono implementati in modo diverso. Un vector è implementato come un array dinamico di elementi contigui in memoria, il che consente un accesso casuale agli elementi dell'array tramite l'operatore di indicizzazione (operator[]). Poiché gli elementi sono memorizzati in modo contiguo, l'accesso agli elementi in un vector è molto efficiente, ma le operazioni di inserimento o cancellazione possono essere costose se devono spostare un grande numero di elementi nell'array per fare spazio per il nuovo elemento o per rimuovere un elemento.

Una list, d'altra parte, è implementata come una lista concatenata di nodi, dove ogni nodo contiene un elemento e un puntatore al nodo successivo. Ciò significa che gli elementi di una list non sono memorizzati in modo contiguo, ma sono collegati tra loro tramite puntatori. L'accesso agli elementi in una list non è casuale, ma è sequenziale, il che significa che per accedere a un elemento bisogna attraversare tutti gli elementi precedenti. Le operazioni di inserimento e cancellazione in una list sono molto efficienti, poiché non richiedono lo spostamento di grandi blocchi di memoria, ma richiedono l'allocazione di un nuovo nodo e la modifica dei puntatori. Tuttavia, la list può essere meno efficiente in termini di utilizzo della memoria rispetto al vector, poiché ogni nodo nella lista richiede un po' di spazio extra per memorizzare il puntatore al nodo successivo.

In sintesi, un vector è preferibile quando è necessario un accesso casuale e efficiente agli elementi e le operazioni di inserimento e cancellazione sono relativamente rare. Una list è preferibile quando sono necessarie operazioni efficienti di inserimento e cancellazione e l'accesso casuale non è necessario o non è importante. In generale, la scelta tra vector e list dipende dalle specifiche esigenze dell'applicazione.